Práctica 05.
Bloque Administración Linux.
Gestionando el software



Autor: Francisco de Asís Conde Rodríguez / Lina García Cabrera
Copyright: Creative Commons Reconocimiento-NoComercial-SinObraDerivada 4.0 Internacional (CC BY-NC-ND 4.0)



Cubre el objetivo del Linux Professional Institute (LPI) 1.102.4. Dependencias, paquete software, Depósito software, Base de datos, Checksums; Sistemas de paquetes: RPM y Debian; Depósito o almacén Software: Categorías; El archivo `/etc/apt/sources.list`, interpretación de los campos; Órdenes para instalar software en Debian; Estado de un paquete software instalado y deseado, estado de error; Cómo instalar / desinstalar un paquete software; Actualización del sistema

Contenidos

Contenidos **1**
1. Introducción **2**
2. Paquetes de software **2**
2.1. Conceptos importantes sobre sistemas de paquetes 3
2.2. Depósitos o almacenes de software 5
3. El sistema de paquetes Debian **5**
3.1. Herramientas del sistema de paquetes Debian 9
¿Cuál es la diferencia entre apt y dpkg? 10
3.2. Comprobando el software instalado 11
3.3. Instalando paquetes 15
3.4. Eliminando paquetes 17
3.5. Actualizando el sistema 19
Ejemplo instalación paso a paso de un paquete Debian **19**
Ejercicios **23**

1. Introducción

Por defecto, un sistema Linux viene con cientos de programas instalados, sin embargo, es frecuente que, como usuario de Linux, tengas que instalar nuevos programas.

En este guion de prácticas se **explica cómo instalar nuevos programas**.

Para ello primero hay que familiarizarse con el concepto de **paquete de software** (***software package***).

En Ubuntu 26.04 LTS. Ubuntu 26.04 LTS (Resolute Raccoon) trae APT 3, que presenta las operaciones con un «Resumen:» en lugar de las líneas «Se instalarán…», y `sudo-rs` en lugar de sudo. La cabecera de `dpkg -l` aparece en inglés. El paquete neofetch del apartado de ejercicios no existe: su equivalente es `fastfetch`. Los ejemplos de este guion se han obtenido en Ubuntu 24.04 LTS, la versión del aula.

2. Paquetes de software

Los programas en Linux son muy **modulares**. Eso significa que raramente un sólo programa contiene todo lo que necesita para ejecutarse. La mayoría de los programas tienen dependencias.

Las **dependencias** son un conjunto de software, que tiene que estar instalado también en el sistema, para que un programa dado pueda funcionar bien.

La mayoría de las distribuciones actuales de sistemas operativos basados en Linux instalan software en paquetes precompilados, que son archivos que contienen binarios de software, archivos de configuración e información sobre dependencias. Un **paquete de software** es un fichero (a veces grande) que contiene el producto a instalar y unas reglas. Estas reglas pueden ser múltiples:

- **Gestión de las dependencias**: sólo se podrá instalar el producto si los productos que él mismo utiliza están ya presentes.
- **Preinstalación**: se deben prever acciones antes de poder instalar el producto (cambiar derechos, crear directorios, etc.).
- **Postinstalación**: se deben prever acciones después de la instalación del producto (parámetros de un fichero de configuración, compilación anexa, etc.).

Gestionar bien las dependencias de los programas es uno de los grandes retos de administrar un sistema Linux. Al instalar un programa, se pueden obtener mensajes de error indicando que para instalar el programa A es necesario tener instalado el programa B

o las bibliotecas C y D. Esto se conoce como el **infierno de las dependencias** (dependency hell), y es una de las causas por las que en el pasado, los usuarios noveles no usaban Linux.

Hoy en día, prácticamente todas las distribuciones de Linux incluyen algún mecanismo para gestionar las dependencias. Todos ellos se basan en los conceptos de **paquete de software** (software package), y **depósito o almacén de software** (software repository).

Al mecanismo para gestionar el software instalado en una distribución Linux se le denomina **sistema de paquetes**.

Aunque en la actualidad la gestión de dependencias sigue siendo el mecanismo habitual para instalar o mantener actualizados los componentes de un sistema operativo basado en Linux, también existen otras **iniciativas alternativas como Flatpak, Snap[1] o AppImage** para solventar el problema del infierno de las dependencias. Estos sistemas, que pueden coexistir con un gestor de paquetes convencional[2], también disponen de sus propios depósitos o almacenes de software, pero a diferencia de los gestores de dependencias convencionales, cuando instalan el paquete de una aplicación, este paquete incluye todas las dependencias que necesita para su funcionamiento, aunque ya existan otras aplicaciones instaladas que tengan dependencias iguales. Además, las aplicaciones instaladas de esta forma suelen ejecutarse en entornos restringidos que eviten interacciones que puedan afectar a otras aplicaciones del sistema. La ventaja principal de este tipo de aplicaciones es que es mucho más sencillo para los desarrolladores de las aplicaciones publicar versiones de sus productos que puedan ejecutarse en cualquier distribución de Linux. Sin embargo, puesto que las aplicaciones instaladas de esta forma aportan sus propias dependencias, esto supondrá en muchos casos redundancia de dependencias similares en disco e incluso en memoria, puesto que dos aplicaciones independientes podrían tener cargadas en memoria las mismas bibliotecas que necesitan su funcionamiento. Por lo tanto, podemos considerar que este tipo de iniciativas son más adecuadas para aplicaciones destinadas a usuarios finales, mientras que en instalaciones de Linux concebidas para albergar servicios y, por lo tanto, donde el uso eficiente de los recursos es prioritario, se sigue trabajando mayoritariamente con gestores de dependencias convencionales.

2.1. Conceptos importantes sobre sistemas de paquetes

Antes de aprender a **manejar los sistemas de paquetes de software**, es muy importante estudiar algunos conceptos relacionados con ellos:

- **Paquete**. Un paquete de software es una **colección de archivos que se instalan juntos en el ordenador**. Incluye programas, archivos auxiliares, archivos de

[1] Actualmente no existe un consenso claro sobre la mejor forma de abordar este tipo de iniciativas y existen varias formas de enfocar el problema, cada una con sus ventajas e inconvenientes, dependiendo de la organización que la respalde, e.g. FlatPak es una iniciativa de RedHat, mientras que Snap es una iniciativa de Canonical (Ubuntu)

[2] La herramienta gráfica de instalación de software de Ubuntu permite seleccionar al usuario el tipo de instalación de un programa si este está disponible tanto en el gestor de dependencias como en algún otro repositorio de software basado en snap o flatpak si estos sistemas están instalados en el sistema operativo.

configuración, etc. Por lo general, se distribuyen como un único archivo que es parecido a un archivo **`tar`** (tarball) o **`zip`**.

Cuando se instala un paquete de software, decenas e incluso cientos de archivos se copian al ordenador y después se ejecuta un script o programa de configuración para realizar de forma automática la configuración del programa.

El **sistema de paquetes** lleva cuenta de todos los **archivos instalados**, el **paquete al que pertenecen** y **el directorio en el que se encuentran**.

- **Base de datos de archivos instalados**. El sistema de paquetes mantiene una base de datos de todos los archivos instalados. La base de datos incluye información sobre:
    - cada fichero instalado a través del sistema de paquetes,
    - el nombre del paquete al que pertenece,
    - e información asociada adicional.
- **Dependencias**. Uno de los tipos de información más importante, mantenido por el sistema de paquetes, es la información sobre dependencias, es decir, las necesidades que tiene un paquete de software de que esté instalado otro paquete de software para funcionar correctamente.

  Por ejemplo, si `MiPrograma` necesita a `MiBibliotecaDeArbolesB` para realizar su trabajo, esta información está en la base de datos de paquetes. Así, si se intenta instalar `MiPrograma` sin que esté instalada `MiBibliotecaDeArbolesB`, el sistema de paquetes lo avisará e impedirá (también puede hacerse que el propio sistema de paquetes resuelva el problema). De forma similar, si se intenta desinstalar `MiBibliotecaDeArbolesB` estando todavía instalado `MiPrograma`, el sistema de paquetes lo avisará e impedirá.

- Checksums[3]. Los checksums permiten **verificar la validez del software instalado**. Un **checksum es un número que se obtiene a partir de un archivo**. Si debido a un error de disco un archivo ha sido dañado o modificado, su checksum no coincidirá con el que guarda el sistema de paquetes. La comprobación no es automática: se solicita con `dpkg --verify` (o `dpkg -V`), que lista los ficheros modificados, y el paquete puede reinstalarse con `apt install --reinstall`.
- **Actualización y desinstalación de software**. El sistema de paquetes lleva cuenta de los archivos y sus dependencias, por lo que permite una actualización a una nueva versión o una desinstalación de forma sencilla.

  Por ejemplo, al desinstalar un paquete, se desinstalan todos los archivos asociados, sin que el administrador tenga que preocuparse de cuáles son o dónde se encuentran. Asimismo, si los archivos de un paquete son necesarios para que otro funcione correctamente, el sistema no permitirá su desinstalación.

[3] Un **checksum**, o **suma de comprobación**, es el resultado de la ejecución de un algoritmo dentro de un archivo único, función denominada *Cryptographic hash function*. Comparar el *checksum* que generas desde tu versión del archivo, junto al provisto por la fuente del mismo, representa una ayuda para asegurarte una copia genuina y libre de errores.

Los dos **sistemas de paquetes** más usuales son ***R**ed hat **P**ackage Manager* (***RPM***) y ***Debian Package System***. Ambos son sistemas de paquetes, por tanto proporcionan la misma funcionalidad, sin embargo **son incompatibles entre sí**, ya que usan distinto formato para los paquetes (.rpm y .deb respectivamente) y la base de datos de archivos instalados. **No se puede instalar directamente un paquete `rpm` en un sistema Debian y viceversa**.

Ambos sistemas de paquetes pueden instalarse en un mismo sistema Linux, pero **es desaconsejable**, ya que **tienen bases de datos de paquetes instalados separadas**. Si se instala un programa usando RPM y a continuación se instala ese mismo programa usando *Debian Package System*, como las bases de datos son distintas, Debian no se dará cuenta de que ese programa ya se ha instalado y viceversa. Eso puede dejar el sistema en una situación inestable o errónea.

Esta es la razón por la que la mayoría de las distribuciones de Linux optan por instalar uno sólo de los dos sistemas de paquetes.

- La distribución **Red Hat** y las que se basan en ella como Fedora, Rocky Linux o Alma Linux, usan el sistema de paquetes rpm. También lo usa openSUSE, que no deriva de Red Hat.
- La distribución **Debian** y las que se basan en ella como **Ubuntu**, Linux Mint, Elementary OS entre otras, usan el sistema de paquetes Debian.

2.2. Depósitos o almacenes de software

Un **depósito o almacén de software** (*software repository*) es una **lista o catálogo de paquetes instalables**. Permite a un sistema de paquetes saber cual es el software disponible para instalar o actualizar. Normalmente se encuentra en un servidor de confianza de la empresa o grupo que realiza la distribución de Linux que se está usando, aunque también puede estar en un CD o DVD de instalación.

Así por ejemplo, en la dirección `http://archive.ubuntu.com` se encuentra el depósito principal de Ubuntu, y en `http://security.ubuntu.com`, el que contiene solo las actualizaciones de seguridad.

Los depósitos de software también permiten descargar directamente los distintos paquetes que se quieran instalar.

La **gestión de los depósitos de software** la realiza directamente **el sistema de paquetes**, y por tanto, el administrador no accede a ellos directamente sino a través de las herramientas que proporciona el sistema de paquetes.

3. El sistema de paquetes Debian

En las **distribuciones basadas en Debian como Ubuntu, el sistema de paquetes que se usa es el *Debian Package System***. Este sistema de paquetes mantiene una lista de todos los depósitos de software disponibles. Tradicionalmente estaba en el archivo

`/etc/apt/sources.list`; desde Ubuntu 24.04, la instalación la escribe en `/etc/apt/sources.list.d/ubuntu.sources`, y `sources.list` solo contiene un comentario que remite a ese fichero. Los depósitos de software más importantes se añaden automáticamente a este archivo durante la instalación del sistema operativo, sin embargo también puede editarse para añadir manualmente otros depósitos de software para que estén disponibles para el sistema de paquetes de Debian.

En Ubuntu, los paquetes de software se clasifican en **categorías** que se refieren al estado del paquete, por ejemplo, si el paquete se considera seguro o si tiene una licencia de uso que no corresponde con la common open source standards.

Las **categorías que se usan en Ubuntu** son:

- **Main**. Esta categoría contiene todo el software que es soportado oficialmente por *Canonical*[4], la compañía que está detrás de Ubuntu. El software que se instala normalmente en una distribución Ubuntu pertenece a esta categoría. Si sólo se instala software de esta categoría, nos aseguramos de tener un sistema estable y, lo más importante para las empresas, que va a tener soporte técnico.
- **Restricted**. Esta categoría contiene el software que, si bien es soportado oficialmente, usa una licencia de uso que no es libre. Incluye los controladores (*drivers*) de algunos dispositivos (como las tarjetas gráficas de NVidia, tarjetas de red, paneles táctiles) o el software que hay que pagar.

  Por lo general, el software de esta categoría se encuentra en un subdirectorio específico del depósito de software.

- **Universe**. Esta categoría contiene software libre que no es soportado oficialmente. Se puede usar, y por lo general no causa problemas, pero no es posible recibir soporte técnico por parte de *Canonical* para esta categoría de software.
- **Multiverse**. Esta categoría contiene software no libre que no es soportado oficialmente.
- **Backports.** No es una categoría, sino un depósito adicional (`noble-backports`) con versiones más recientes de algunos programas, adaptadas desde versiones posteriores de Ubuntu. Contiene paquetes de las cuatro categorías anteriores, y APT no instala nada de él salvo que se le pida expresamente.

Cuando se instala software usando las herramientas del sistema de paquetes, éstas buscan la información de los paquetes disponibles y sus dependencias en las fuentes de instalación (los depósitos de software) que se listan en el archivo **`/etc/apt/sources.list`** o, en distribuciones modernas y también para repositorios de terceros, en archivos similares en la carpeta **`/etc/apt/sources.list.d/`** con extensiones **`.list`** o **`.sources`**

[4] **Canonical** tiene una tradición de otorgar un **nombre clave a cada versión de Ubuntu** compuesto del **nombre de un animal y un adjetivo**, empezando con Ubuntu 4.10 Warty Warthog. Desde Ubuntu 6.06 Dapper Drake, las iniciales van en orden alfabético, y por eso Ubuntu 14.10 fue Wily Werewolf, y 16.04 es Xenial Xerus. Xenial Xerus significa “Xerus Hospitalario”, siendo el xerus una especie de ardilla terrestre propia de África. El **primer número se refiere al año en el que fue lanzado**, y el **segundo número se refiere al mes**. La primera versión de Ubuntu fue la 4.10

El formato de estos archivos está actualmente en periodo de migración del **antiguo formato `.list`** a un **nuevo formato `.sources`** y podemos encontrarnos los dos tipos, aunque el contenido es similar en ambos casos y totalmente reconocido en distribuciones nuevas basadas en Debian. Por ejemplo, en el siguiente listado pueden verse tres de las líneas de un archivo con extensión *`.list`* en la distribución Ubuntu Linux 12.04 LTS en el formato anterior:

```
deb cdrom:[Ubuntu 12.04 LTS _Precise Pangolin_ - Release amd64 (20120425)]/
precise main restricted
deb http://es.archive.ubuntu.com/ubuntu/ precise main restricted
deb-src http://es.archive.ubuntu.com/ubuntu/ precise main restricted
```

Un ejemplo de formato moderno *.sources* de fichero de definición de repositorios de paquetes en Ubuntu 24.04 es:

```
Types: deb
URIs: http://archive.ubuntu.com/ubuntu/
Suites: noble noble-updates noble-backports
Components: main restricted universe multiverse
Signed-By: /usr/share/keyrings/ubuntu-archive-keyring.gpg
```

La versión de Ubuntu instalada en las aulas es la **Ubuntu 24.04 LTS** (LTS significa “Long Term Support”) cuyo nombre es “**Noble Numbat**”. Este nombre en clave sigue la tradición de Ubuntu de usar un adjetivo y un nombre de animal, y "Noble Numbat" hace referencia a un marsupial australiano.

Ubuntu 22.04 LTS que se llama "Jammy Jellyfish", «medusa de mermelada».

La versión de Ubuntu 20.04 LTS que se llama "Focal Fossa".

La versión que se usará en casa, Ubuntu 26.04 LTS, se llama "Resolute Raccoon" (en los ficheros de configuración, resolute).

Cada línea tiene cuatro o más campos. Entre el primero y el segundo puede aparecer, entre corchetes, una lista de opciones, como `[arch=amd64 signed-by=/usr/share/keyrings/clave.gpg]`.

- El primero indica el **formato de los paquetes** que se encuentran en ese depósito de software. Hay dos tipos **`deb`** y **`deb-src`**. El primero indica **paquetes binarios** de software mientras que el segundo **paquetes de código fuente**. En la imagen tiene el número 2.

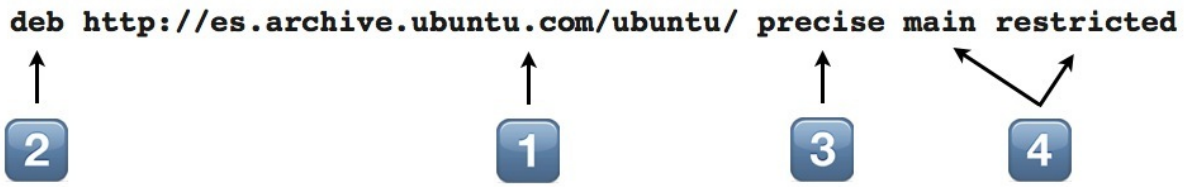
  Los **paquetes binarios** incluyen los programas ya compilados y enlazados, listos para ejecutarse en la distribución de Linux donde se instalan. Por el contrario, los **paquetes de código fuente** incluyen los archivos de código fuente a partir de los cuales se pueden construir los binarios de los programas compilándolos y enlazándolos. La principal ventaja de los paquetes binarios es su comodidad, ya que se instalan directamente sin necesidad de compilación previa, a diferencia de los paquetes de código fuente, que requieren disponer de herramientas de construcción y realizar el proceso de compilación antes de su instalación.

- El segundo campo es **la dirección donde se encuentra el depósito de software**. Normalmente es la URL de un servidor en internet (como en las dos últimas líneas del ejemplo), pero también puede hacer referencia al CD de instalación (como en la primera línea) o incluso a un directorio de nuestro equipo en el que se haya descargado una copia del depósito de software. En la imagen aparece con el número 1.
- El tercer campo hace referencia al **nombre de la distribución** (en el ejemplo la distribución es la 12.04 LTS que tiene como seudónimo: `precise pangolin` o más abreviadamente `precise`). En la imagen aparece con el número 3.
- Los campos cuarto y siguientes hacen referencia a la **categoría o categorías de paquetes** que pueden encontrarse en el depósito de software. en la imagen aparece con el número 4.

  Por ejemplo, en el CD de instalación sólo se encuentran paquetes de las categorías `main` y `restricted`.

  En el formato moderno (.sources), los mismos datos se escriben como campos con nombre: Types (campo 1), URIs (campo 2), Suites (campo 3, que admite varias distribuciones a la vez) y Components (campos 4 y siguientes). Signed-By indica la clave con la que se comprueba la firma del depósito.

La forma correcta de interpretar cada entrada del archivo /etc/apt/sources.list puede verse en el siguiente ejemplo:



1. En el depósito de software cuya URL es http://es.archive.ubuntu.com/ubuntu/
2. pueden encontrarse paquetes binarios
3. pertenecientes a la distribución precise pangolín (12.04 LTS)
4. pertenecientes a las categorías main y restricted.

Depósitos configurados en Ubuntu 24.04

```
alumno@so-lab01:~$ cat /etc/apt/sources.list
# Ubuntu sources have moved to the /etc/apt/sources.list.d/ubuntu.sources
# file, which uses the deb822 format. Use deb822-formatted .sources files
# to manage package sources in the /etc/apt/sources.list.d/ directory.
# See the sources.list(5) manual page for details.
alumno@so-lab01:~$ grep -v -e '^#' -e '^$' /etc/apt/sources.list.d/ubuntu.sources
Types: deb
URIs: http://archive.ubuntu.com/ubuntu/
Suites: noble noble-updates noble-backports
Components: main universe restricted multiverse
Signed-By: /usr/share/keyrings/ubuntu-archive-keyring.gpg
Types: deb
URIs: http://security.ubuntu.com/ubuntu/
Suites: noble-security
Components: main universe restricted multiverse
Signed-By: /usr/share/keyrings/ubuntu-archive-keyring.gpg
```

Nombres de los Paquetes

Los nombres de los paquetes de Debian se forman de la siguiente manera:

```
nombrePaqueteDebian_Numeroversion-Numerorevision_arquitectura.deb
```

por ejemplo

```
zsh_5.9-6ubuntu2_amd64.deb
```

El `número` dice la **versión** y el **número de revisión** del paquete software Debian que contiene, mientras que el `nombre de la arquitectura` especifica la **arquitectura de computadores (amd64, arm64, o all para los paquetes que no dependen de ella, como los de documentación).**

El ejemplo anterior corresponde al paquete `zsh`, versión 5.9 del programa, revisión 6 del paquete en Debian y segunda modificación de Ubuntu sobre él (ubuntu2), para la arquitectura amd64.

3.1. Herramientas del sistema de paquetes Debian

El sistema de paquetes Debian proporciona dos herramientas para la **gestión del software instalado** en el ordenador:

- la suite de herramientas **APT** (***A**dvanced **P**ackaging **T**ool*) y
- la orden **`dpkg`** (***D**ebian **P**ac**K**a**G**e tool*).

Estas son herramientas de la línea de órdenes que permiten una gestión simple del software instalado, aunque también existen herramientas con una interfaz más elaborada tales como **`dselect`**, **`aptitude`** y **`synaptic`**.

De entre todas las herramientas que proporciona la suite **APT**, la más utilizada es **`apt-get`** o **`apt`**[5]. Esta orden obtiene la lista de depósitos de `/etc/apt/sources.list` y de `/etc/apt/sources.list.d/`, y la de paquetes instalados, de la base de datos de `dpkg`

[5] A partir de Ubuntu 14.04 se puede sustituir "**`apt-get`**" por simplemente "**`apt`**".

(`/var/lib/dpkg/status`). Las acciones que modifican el sistema (install, remove, update…) exigen permisos de `root` y se preceden de `sudo`; las de consulta (`list`, `show`, `search`) no. Su formato es:

```
$ apt [opciones] [acción] [nombres de paquetes]
```

La orden **`dpkg`** toma el siguiente formato:

```
$ dpkg [opciones] [acción] [archivo de paquete | nombre de paquete]
```

Un **archivo de paquete** es un archivo en disco con extensión `.deb`, que contiene un paquete binario para ser instalado en el sistema (los paquetes de código fuente no son un único fichero, sino un fichero `.dsc` con uno o varios archivos comprimidos). Un nombre de paquete es un nombre simbólico que corresponde total o parcialmente con el nombre de algún paquete ya instalado en el sistema o bien con el nombre de algún paquete del que el sistema de paquetes tiene conocimiento.

¿Cuál es la diferencia entre apt y dpkg?

`APT` y **`dpkg`** son interfaces de administración de paquetes de línea de comandos que se pueden usar en la terminal en Ubuntu y otros sistemas basados en Debian. Pueden, entre otras cosas, **instalar archivos DEB y enumerar los paquetes instalados**.

Si son tan similares, ¿por qué necesita tanto `APT` como `dpkg`?

Porque las dos interfaces funcionan juntas, y **APT actúa como una herramienta completa de administración de paquetes mediante el uso de dpkg**.

APT utiliza dpkg para instalar paquetes

Cuando **`APT`** (o su primo, `apt-get`) instala un paquete, en realidad está usando `dpkg` en el back-end para lograrlo. De esa manera, `dpkg` actúa más como una herramienta "oculta" para la interfaz más amigable de APT.

APT puede descargar paquetes

Con APT, se puede recuperar un archivo de un repositorio remoto e instalarlo, todo en un solo comando. Esto le ahorra el trabajo de buscar y descargar manualmente el paquete antes de la instalación.

Con **`dpkg`**, **solo se puede instalar archivos locales** que ya se hayan descargado. No se puede buscar repositorios remotos ni extraer paquetes de ellos.

Dpkg no instalará dependencias

Cuando se instala un paquete con **dpkg**, eso es todo lo que sucederá: el sistema simplemente instalará el paquete. Sin embargo, algunos paquetes necesitan un software adicional llamado dependencias para funcionar. Si este es el caso, **dpkg** podría advertirle con un mensaje de error.

APT, sin embargo, buscará y obtendrá automáticamente las dependencias relevantes para asegurarse de que lo que esté intentando instalar funcione correctamente. Es por eso que recomendamos que, después de instalar un paquete con **dpkg**, siga usando el comando especial de APT para restaurar dependencias. También se puede instalar un fichero `.deb` local directamente con APT, que resuelve las dependencias en el mismo paso: `sudo apt install ./fichero.deb` (la ruta, con «./», indica que es un fichero y no el nombre de un paquete).

```
alumno@so-lab01:~$ sudo apt install -f
```

Dpkg indexa solo paquetes locales

Si se está tratando de averiguar sobre los paquetes instalados en su sistema, la función **`--list`** en `dpkg` podría ser mejor que APT. Eso es porque, de la misma manera que no se puede encontrar y descargar paquetes remotos, `dpkg` tampoco puede listar paquetes que no sean locales para el dispositivo.

```
alumno@so-lab01:~$ dpkg --list
```

El comando list de APT enumerará todos los paquetes que conoce, locales o de otro tipo.

```
alumno@so-lab01:~$ apt list
```

Para ver solo los paquetes instalados, debe pasar la opción `--installed` o `-i` .

```
alumno@so-lab01:~$ apt list --installed
```

Si se quiere contar los paquetes instalados en Ubuntu, ¿cómo lo harías usando el comando `wc` con `apt`?

3.2. Comprobando el software instalado

La forma de **comprobar el software instalado** en el ordenador es con la orden **`dpkg -l`** que genera un **largo listado con todos los paquetes instalados actualmente en el ordenador**. Cuando la salida es el terminal, dpkg -l la muestra con un paginador (como `less`): se avanza con la barra espaciadora y se sale con `q`.

El siguiente listado es una parte del resultado mostrado por la orden **`dpkg -l`**:

```
alumno@so-lab01:~$ dpkg -l
Deseado=desconocido(U)/Instalar/eliminaR/Purgar/retener(H)
|
Estado=No/Inst/ficheros-Conf/desempaqUetado/medio-conF/medio-inst(H)/espera-dispa
ro(W)/pendienTe-disparo
|/ Err?=(ninguno)/requiere-Reinst (Estado,Err: mayúsc.=malo)
||/ Nombre                      Versión                          Arquitectura
Descripción
+++-==========================-================================-============-==
==============================================================================
ii  adduser                    3.137ubuntu1                     all
add and remove users and groups
ii  apt                        2.8.3                            amd64
commandline package manager
ii  apt-utils                  2.8.3                            amd64
package management related utility programs
ii  base-files                 13ubuntu10.5                     amd64
Debian base system miscellaneous files
ii  base-passwd                3.6.3build1                      amd64
Debian base system master password and group files
ii  bash                       5.2.21-2ubuntu4                  amd64
GNU Bourne Again SHell
…
```

La primera columna se refiere al **estado del paquete**. Consta de **tres caracteres**:

1. El **primero** indica el **estado deseado del paquete**, es decir, el estado en el que debería encontrarse el paquete dada su historia de instalación/desinstalación.
2. El **segundo** indica el **estado actual del paquete**, es decir, el estado real en el que se encuentra.
3. Y el **tercero** indica cualquier **estado de error** conocido asociado con el paquete. Por lo general es el **espacio en blanco**, lo que indica que **no hay errores**.

El primer carácter indicador de estado "**estado deseado**" puede tomar cualquiera de los siguientes valores:

- **`i`**: (***install)*** Indica que el paquete **debería estar instalado**. Es el indicador que se muestra en la mayoría de los casos.
- **`h`**: (***hold***) Indica que el paquete **no puede ser modificado**.
- **`p`**: (***purge***) Indica que el paquete **debería haberse eliminado por completo**, incluyendo sus archivos de configuración.
- **`r`**: (***remove***) Indica que el paquete **debería haberse eliminado, sin eliminar los archivos de configuración asociados**.
- **`u`**: (***unknown***) Indica que **no hay un estado deseado** conocido para este paquete.

El segundo carácter indicador de estado "**estado actual**" puede tomar cualquiera de los siguientes valores:

- **`i`**: (***Installed***) Indica que el paquete está **realmente instalado**.
- **`c`**: (***Config-file***s) Indica que **los archivos de configuración del paquete están instalados**, pero el resto de archivos del paquete no.

- **`F: (Half-configured)`** Indica que el paquete está instalado pero **no hay garantías de que esté correctamente instalado**.
- **`H: (Half-installed)`** Indica que el paquete está **parcialmente instalado**.
- **`n`**: (***Not-installed***) Indica que el paquete **no está instalado en este momento**.
- **`W:`** (Triggers-awaiting) y **`t:`** (Triggers-pending) Indican que el paquete espera a que se procesen sus disparadores (triggers), acciones aplazadas como actualizar el índice de las páginas del manual.
- **`U: (Unpacked)`** Indica que el paquete se instaló, pero **la instalación no finalizó** ya que el script de configuración que se debe ejecutar tras la ejecución del paquete, no se completó con éxito.

El tercer carácter indicador de estado indica cualquier **estado de error** conocido asociado con el paquete. En la mayoría de los casos se muestra un espacio en blanco, lo que indica que no hay ningún error. Otras opciones para este carácter son:

- **`R`**: (***Reinst-require***d) Indica que es necesario reinstalar el paquete.

La retención (hold) no es un error: se indica con h en el primer carácter (estado deseado), por ejemplo hi, y se activa con sudo apt-mark hold paquete.

La orden **`dpkg -l`** puede llevar como argumento parte del nombre de un paquete (entre comillas simples), en cuyo caso muestra sólo los datos de instalación de los paquetes cuyo nombre coincida con el argumento.

Por ejemplo, la orden **`dpkg -l 'zi*'`** muestra el siguiente listado (en una instalación básica de Ubuntu Linux 24.04 LTS):

```
alumno@so-lab01:~$ dpkg -l 'zi*'
Deseado=desconocido(U)/Instalar/eliminaR/Purgar/retener(H)
|
Estado=No/Inst/ficheros-Conf/desempaqUetado/medio-conF/medio-inst(H)/espera-dispa
ro(W)/pendienTe-disparo
|/ Err?=(ninguno)/requiere-Reinst (Estado,Err: mayúsc.=malo)
||/ Nombre         Versión         Arquitectura Descripción
+++-==============-===============-============-=================================
ii  zip            3.0-13ubuntu0.2 amd64        Archiver for .zip files
```

Otra orden interesante para conocer los archivos instalados de un paquete es **`dpkg -L nombrePaquete`**. Muestra un listado con todos los archivos instalados por ese paquete, en un paquete concreto.

Por ejemplo, la orden **`dpkg -L zip`** muestra el siguiente listado:

```
alumno@so-lab01:~$ dpkg -L zip
/.
/usr
/usr/bin
/usr/bin/zip
/usr/bin/zipcloak
/usr/bin/zipnote
/usr/bin/zipsplit
/usr/share
/usr/share/doc
/usr/share/doc/zip
/usr/share/doc/zip/TODO
/usr/share/doc/zip/WHATSNEW
/usr/share/doc/zip/changelog.Debian.gz
/usr/share/doc/zip/copyright
/usr/share/man
/usr/share/man/man1
/usr/share/man/man1/zip.1.gz
/usr/share/man/man1/zipcloak.1.gz
/usr/share/man/man1/zipnote.1.gz
/usr/share/man/man1/zipsplit.1.gz
/usr/share/doc/zip/changelog.gz
```

Como se puede ver, al instalar un paquete de un programa, no sólo se instalan los binarios de ese programa en el ejemplo: **`/usr/bin/zip`**, sino que también se instalan archivos adicionales como las páginas de manual para el programa: **`/usr/share/man/man1/zip.1.gz`** o información sobre los cambios en la nueva versión: **`/usr/share/doc/zip/WHATSNEW`**.

Una última orden muy útil para **mostrar información de un paquete que ya está instalado en el sistema** es **`dpkg -s nombrePaquete`** o también **`apt show nombrePaquete.`** La primera solo funciona con paquetes instalados; **`apt show`** muestra también los que están en los depósitos sin instalar.

Por ejemplo: la orden **`dpkg -s zip`** muestra el siguiente listado:

```
alumno@so-lab01:~$ dpkg -s zip
Package: zip
Status: install ok installed
Priority: optional
Section: utils
Installed-Size: 536
Maintainer: Ubuntu Developers <ubuntu-devel-discuss@lists.ubuntu.com>
Architecture: amd64
Multi-Arch: foreign
Version: 3.0-13ubuntu0.2
Depends: libbz2-1.0, libc6 (>= 2.34)
Recommends: unzip
Description: Archiver for .zip files
 This is InfoZIP's zip program. It produces files that are fully
 compatible with the popular PKZIP program; however, the command line
 options are not identical. In other words, the end result is the same,
 but the methods differ. :-)
 .
 This version supports encryption.
Homepage: https://infozip.sourceforge.net/Zip.html
Original-Maintainer: Santiago Vila <sanvila@debian.org>
```

Como puede verse, esta orden incluye mucha información interesante como **dependencias**, otros **paquetes recomendados** (paquetes que se deberían instalar junto con este), etc.

3.3. Instalando paquetes

Para instalar nuevos paquetes en el sistema se usa la orden **`apt-get`** (para scripting y uso avanzado) o **`apt`** (recomendado para el usuario final) con la acción `install` seguida del nombre del paquete que se quiere instalar. Como se trata de una orden que debe ejecutarse con permisos de root, debe precederse de `sudo`. Por tanto el formato de la orden para instalar nuevos paquetes queda así:

```
$ sudo apt install nombrePaquete
$ sudo apt-get install nombrePaquete
```

Una de las ventajas de **`apt-get`** es que no sólo instala el paquete sino que lo busca en los depósitos de software disponibles y lo descarga al directorio **`/var/cache/apt/archives/`**. Una vez descargado lo instala y ejecuta el script de post-instalación, con lo que el paquete queda perfectamente instalado y configurado. A diferencia de `apt-get`, la orden apt borra esos ficheros después de instalarlos correctamente, para no ocupar espacio.

Además, **si el paquete tiene dependencias, `apt-get install` descarga e instala también aquellos otros paquetes que sean necesarios para que el paquete que queremos instalar funcione correctamente**.

Por ejemplo, para instalar el paquete de software **`nmap`**, se debe ejecutar la orden: **`sudo apt-get install nmap, lo que produce`** la siguiente salida por pantalla:

```
alumno@so-lab01:~$ sudo apt-get install nmap
[sudo] contraseña para alumno:
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Se instalarán los siguientes paquetes adicionales:
  ibverbs-providers libblas3 libibverbs1 liblinear4 liblua5.4-0 libnl-3-200
libnl-route-3-200
  libpcap0.8t64 libssh2-1t64 nmap-common
Paquetes sugeridos:
  liblinear-tools liblinear-dev ncat ndiff zenmap
Se instalarán los siguientes paquetes NUEVOS:
  ibverbs-providers libblas3 libibverbs1 liblinear4 liblua5.4-0 libnl-3-200
libnl-route-3-200
  libpcap0.8t64 libssh2-1t64 nmap nmap-common
0 actualizados, 11 nuevos se instalarán, 0 para eliminar y 0 no actualizados.
Se necesita descargar 7.299 kB de archivos.
Se utilizarán 30,7 MB de espacio de disco adicional después de esta operación.
¿Desea continuar? [S/n] S
Des:1 http://archive.ubuntu.com/ubuntu noble-updates/main amd64 libnl-3-200 amd64
3.7.0-0.3build1.1 [55,7 kB]
…
Descargados 7.299 kB en 1s (7.934 kB/s)
Seleccionando el paquete libnl-3-200:amd64 previamente no seleccionado.
(Leyendo la base de datos ... 9911 ficheros o directorios instalados
actualmente.)
Preparando para desempaquetar .../00-libnl-3-200_3.7.0-0.3build1.1_amd64.deb ...
Desempaquetando libnl-3-200:amd64 (3.7.0-0.3build1.1) ...
…
Configurando nmap (7.94+git20230807.3be01efb1+dfsg-3build2) ...
Procesando disparadores para man-db (2.12.0-4build2) ...
Procesando disparadores para libc-bin (2.39-0ubuntu8.9) ...
```

Como se puede ver, existen cuatro fases en el proceso de instalación:

1. **Leer la información de los depósitos de software para construir la lista de paquetes disponibles, crear el árbol de dependencias y leer la información de estado actual del paquete que se ha pedido instalar**. Con todo eso, la orden sabe si el paquete está disponible para su instalación y si las dependencias se satisfacen. Si todo está correcto, la orden puede continuar.
2. Ya que se han leído la lista de paquetes y demás información relacionada, la orden **`apt-get` aprovecha para dar algunos mensajes relativos al sistema de paquetes aunque no estén relacionados con el paquete que se está instalando**. En este ejemplo, la orden informa de los paquetes adicionales que necesita nmap (sus dependencias) y de los paquetes sugeridos, que no se instalan; como hay paquetes adicionales, pide confirmación.

   Si no hubiese ningún paquete que necesitase nuestra atención, esta fase no produciría ningún resultado en la pantalla.

3. En esta fase se **descarga el paquete al disco**, en este caso el archivo de paquete binario: `nmap_7.94+git20230807.3be01efb1+dfsg-3build2_amd64.deb`, **junto con los de sus dependencias; después se descomprimen y se instalan**.
4. Por último **se ejecuta el script de post-instalación que configura** `nmap`. Esto hace que el paquete quede perfectamente configurado y listo para usar.

Una opción interesante para la acción `install` de la orden `apt-get` es `-d` o `--download-only`, que sólo descarga el paquete de software al directorio `/var/cache/apt/archives/`, pero no lo instala ni configura. Esta opción es útil, por ejemplo, cuando se quiere instalar un paquete de software en otro ordenador sin conexión a la red. Se descarga el paquete en otro ordenador y de allí se copia en un pendrive para su uso en el ordenador sin conexión.

La orden **`sudo apt-get install -d nmap`** descarga el archivo de paquete binario: `nmap_7.94+git20230807.3be01efb1+dfsg-3build2_amd64.deb` y los de las dependencias que falten en el directorio `/var/cache/apt/archives/`

Órdenes curiosas

Algunas órdenes divertidas, curiosas o útiles que puedes instalar son: **`sl`**, **`tree`**, **`cowsay`**, **`fortune`**, **`bmon`**, **`pi`**, **`rev`**, **`factor`**, **`figlet`**, **`banner`**, **`oneko`**, **`time`**, **`time cat`**, **`ncat`**, **`curl`** (ejemplo de ejecución, `curl wttr.in/Jaen`), `yes`, `aafire`, `cmatrix`, `toilet`,

Recomendación: Más de 400 comandos para GNU/Linux que deberías conocer :D

https://blog.desdelinux.net/mas-de-400-comandos-para-gnulinux-que-deberias-conocer/

¿Te gustaría tener un acuario? Existe una orden que te muestra la Pecera ASCII para cuando estés aburrido,

```
alumno@so-lab01:~$ asciiquarium
```

Intenta averiguar cómo tienes que instalar esta orden.

3.4. Eliminando paquetes

Para eliminar paquetes ya instalados en el sistema, se usa la orden **`apt-get`** con la acción **`remove`** seguida del nombre del paquete que se quiere eliminar. Como se trata de una orden que debe ejecutarse con permisos de root, debe precederse de `sudo`. Por tanto el formato de la orden para eliminar paquetes ya instalados queda así:

```
$ sudo apt-get remove nombrePaquete
```

IMPORTANTE: La orden `apt-get remove` **NO** elimina los archivos de configuración asociados al paquete de software que se desinstala. Para realizar una desinstalación completa, que incluya también los archivos de configuración es necesario usar la opción **`--purge`**.

Por ejemplo, para eliminar el paquete de software `nmap`, se debe ejecutar la orden: **`sudo apt-get remove nmap`** lo que produce la siguiente salida por pantalla:

```
alumno@so-lab01:~$ sudo apt-get remove nmap
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Los paquetes indicados a continuación se instalaron de forma automática y ya no
son necesarios.
  ibverbs-providers libblas3 libibverbs1 liblinear4 liblua5.4-0 libnl-3-200
libnl-route-3-200
  libpcap0.8t64 libssh2-1t64 nmap-common
Utilice «sudo apt autoremove» para eliminarlos.
Los siguientes paquetes se ELIMINARÁN:
  nmap
0 actualizados, 0 nuevos se instalarán, 1 para eliminar y 0 no actualizados.
Se liberarán 4.169 kB después de esta operación.
¿Desea continuar? [S/n] S
(Leyendo la base de datos ... 10871 ficheros o directorios instalados
actualmente.)
Desinstalando nmap (7.94+git20230807.3be01efb1+dfsg-3build2) ...
Procesando disparadores para man-db (2.12.0-4build2) ...
```

Como puede verse, existen cinco fases en el proceso de desinstalación:

1. **Leer la información de los depósitos de software para construir la lista de paquetes disponibles, crear el árbol de dependencias y leer la información de estado actual del paquete que se ha pedido desinstalar.** Con todo eso, la orden sabe si el paquete puede desinstalarse sin que ello afecte a otros paquetes instalados en el sistema. Si todo está correcto la orden puede continuar.
2. Ya que se han leído la lista de paquetes y demás información relacionada, la orden **`apt-get`** aprovecha para dar algunos **mensajes relativos al sistema de paquetes aunque no estén relacionados con el paquete que se está desinstalando**. En este ejemplo, la orden informa de que las dependencias que se instalaron automáticamente con nmap ya no son necesarias, y propone eliminarlas con sudo apt autoremove.

   Si no hubiese ningún paquete que necesitase nuestra atención, esta fase no produciría ningún resultado en la pantalla.

3. En esta fase se pide **confirmación al usuario de que realmente quiere desinstalar los paquetes que se listan**. El usuario debe pulsar la tecla **`s`** para continuar con la desinstalación.
4. **Se eliminan del disco los archivos que componen el paquete de software**. Esto no incluye los archivos de configuración.
5. Por último se **ejecuta el script de post-desinstalación** que completa la eliminación del paquete **`nmap`**. En este ejemplo, se rehace la base de datos de páginas de manual (**`man-db`**) para que ya no estén disponible las del paquete `nmap`.

Una opción interesante para la acción **`remove`** de la orden **`apt-get`** es `-y` o `--yes` o **`--assume-yes`**, que produce automáticamente la respuesta "si", a cualquier pregunta que la orden `apt-get` haga al usuario durante su ejecución. En el caso de la acción **`remove`**, la opción `-y` hace que no se pregunte al usuario si realmente quiere desinstalar el paquete de software.

Si se quisiera una desinstalación completa del paquete **nmap**, la orden correcta es: Las dependencias instaladas automáticamente no se eliminan con nmap: para retirarlas (con su configuración) se usa después sudo apt autoremove --purge.

```
alumno@so-lab01:~$ sudo apt-get remove --purge nmap
```

3.5. Actualizando el sistema

Una de las aplicaciones más usadas de la orden **apt-get** es actualizar el sistema operativo para mantener la última versión de los distintos programas que lo forman.

Para ello se usan las acciones **update** y **dist-upgrade** ejecutadas en secuencia. Con la orden apt, la segunda acción se llama full-upgrade (sudo apt full-upgrade); apt upgrade actualiza los paquetes sin instalar ni eliminar ninguno.

```
alumno@so-lab01:~$ sudo apt-get update
alumno@so-lab01:~$ sudo apt-get dist-upgrade
```

La primera acción, **update**, **actualiza la lista de paquetes disponibles** a partir de la información contenida en los depósitos de software que se definen en /etc/apt/sources.list y en /etc/apt/sources.list.d/. Es muy importante ejecutar esta orden antes de actualizar el sistema operativo, para que el sistema de paquetes cuente con la información más actualizada sobre paquetes y dependencias.

La segunda acción, **dist-upgrade**, **comprueba aquellos paquetes instalados para los que existen versiones más actualizadas** en los depósitos de software, se asegura de que las actualizaciones no generan problemas de dependencias, descarga los archivos de paquete que se necesiten y los ejecuta en el mejor orden para evitar conflictos.

Dependiendo de la velocidad de la conexión a internet y de si hace mucho tiempo desde la última actualización, esta orden puede tardar bastantes minutos, pero cuando termina tenemos el sistema operativo completamente actualizado.

Ejemplo instalación paso a paso de un paquete Debian

Para terminar el estudio de los sistemas de paquetes veamos un ejemplo paso a paso de cómo gestionar la **instalación / desinstalación de un paquete de software**.

Para ello vamos a instalar la shell **zsh** que por defecto no se instala en la distribución Ubuntu Linux 24.04 LTS.

Antes de comenzar la instalación de nuevos paquetes, es imprescindible actualizar la información local de los repositorios del sistema.

Esto garantiza que el gestor de paquetes disponga de la lista más reciente de versiones y dependencias, evitando errores y asegurando que se instale la versión más actualizada disponible.

```
alumno@so-lab01:~$ sudo apt update
Obj:1 http://security.ubuntu.com/ubuntu noble-security InRelease
Obj:2 http://archive.ubuntu.com/ubuntu noble InRelease
Obj:3 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Obj:4 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Todos los paquetes están actualizados.
alumno@so-lab01:~$ sudo apt upgrade
Leyendo lista de paquetes... Hecho
Creando árbol de dependencias... Hecho
Leyendo la información de estado... Hecho
Calculando la actualización... Hecho
0 actualizados, 0 nuevos se instalarán, 0 para eliminar y 0 no actualizados.
```

1. En primer lugar comprobamos que la shell zsh no está ya instalada en el sistema. Para ello pueden usarse o bien la orden:

```
alumno@so-lab01:~$ dpkg -l zsh
dpkg-query: no se ha encontrado ningún paquete que corresponda con zsh.
```

O bien la orden:

```
alumno@so-lab01:~$ dpkg -L zsh
dpkg-query: el paquete `zsh' no está instalado.
Use dpkg --contents (= dpkg-deb --contents) to list archive files contents.
```

En ambos casos, si el paquete que contiene la shell **`zsh`** no está instalado, se recibe un mensaje indicándolo.

2. Una vez comprobado que el paquete no está ya instalado, podemos instalarlo. Para ello usaremos la orden:

```
alumno@so-lab01:~$ sudo apt-get install zsh
```

3. Ahora podemos comprobar su estado con la orden:

```
alumno@so-lab01:~$ dpkg -l zsh
Deseado=desconocido(U)/Instalar/eliminaR/Purgar/retener(H)
|
Estado=No/Inst/ficheros-Conf/desempaqUetado/medio-conF/medio-inst(H)/espera-dispa
ro(W)/pendienTe-disparo
|/ Err?=(ninguno)/requiere-Reinst (Estado,Err: mayúsc.=malo)
||/ Nombre         Versión      Arquitectura Descripción
+++-==============-============-============-=================================
ii  zsh            5.9-6ubuntu2 amd64        shell with lots of features
```

El estado es **`ii`** lo que indica que no hay errores, que el paquete debería estar instalado y que realmente lo está.

4. Ahora podemos comprobar los archivos que se han instalado en el disco al instalar el paquete `zsh`. Para ello usamos la orden:

```
alumno@so-lab01:~$ dpkg -L zsh
```

que produce un listado muy largo de archivos, entre ellos **`/bin/zsh`** que es el binario mediante el cual se puede ejecutar la shell recién instalada.

Ejecutando la orden **`ls -al /bin/zsh*`** obtenemos como resultado:

```
alumno@so-lab01:~$ ls -al /bin/zsh*
-rwxr-xr-x 1 root root 976304 abr  8  2024 /bin/zsh
-rwxr-xr-x 1 root root    852 feb 23  2024 /bin/zsh5
```

Es decir, que también se puede ejecutar la shell mediante la orden **`zsh`** o zsh5. Cuando instalas `zsh`, el paquete principal instala el binario en /usr/bin/zsh (el intérprete real). Además, Ubuntu mantiene zsh5, un guion de compatibilidad (no un binario compilado) que avisa de que desaparecerá y ejecuta `zsh`.

Vamos a probar ahora la nueva shell instalada. Para ello usamos la orden `zsh` o `zsh5`. Si ejecutas `zsh5`, te advierte que desaparecerá en futuras versiones de Ubuntu. La primera vez que se ejecuta `zsh`, al no existir ningún fichero de configuración del usuario, aparece el menú de `zsh-newuser-install`; con la tecla `q` se sale sin crear nada y aparece el prompt de la nueva shell:

```
alumno@so-lab01:~$ zsh
This is the Z Shell configuration function for new users,
zsh-newuser-install.
You are seeing this message because you have no zsh startup files
(the files .zshenv, .zprofile, .zshrc, .zlogin in the directory
~).  This function can help you with a few settings that should
make your use of the shell easier.

You can:

(q)  Quit and do nothing.  The function will be run again next time.

(0)  Exit, creating the file ~/.zshrc containing just a comment.
     That will prevent this function being run again.

(1)  Continue to the main menu.

(2)  Populate your ~/.zshrc with the configuration recommended
     by the system administrator and exit (you will need to edit
     the file by hand, if so desired).

--- Type one of the keys in parentheses --- q
so-lab01% echo $ZSH_VERSION
5.9
so-lab01% exit
```

Podemos teclear algunas órdenes para probar el funcionamiento de la nueva shell instalada y para salir la orden interna **`exit`**.

5. Ahora vamos a desinstalar la shell **`zsh`**. Para ello ejecutamos la orden:

```
alumno@so-lab01:~$ sudo apt-get remove zsh
```

o la orden:

```
alumno@so-lab01:~$ sudo apt-get -y remove zsh
```

si no queremos que el sistema de paquetes nos pregunte si realmente queremos desinstalar el paquete.

Si intentamos ejecutar la shell **`zsh`** el sistema nos responde con el siguiente mensaje: (si `zsh` ya se había ejecutado en ese mismo terminal; en un terminal nuevo, el sistema responde «No se ha encontrado la orden «zsh», pero se puede instalar con: `sudo apt install zsh`»)

```
alumno@so-lab01:~$ zsh
bash: /usr/bin/zsh: No existe el archivo o el directorio
```

Y por último, si comprobamos el estado mediante la orden:

```
alumno@so-lab01:~$ dpkg -l zsh
Deseado=desconocido(U)/Instalar/eliminaR/Purgar/retener(H)
|
Estado=No/Inst/ficheros-Conf/desempaqUetado/medio-conF/medio-inst(H)/espera-dispa
ro(W)/pendienTe-disparo
|/ Err?=(ninguno)/requiere-Reinst (Estado,Err: mayúsc.=malo)
||/ Nombre         Versión      Arquitectura Descripción
+++-==============-============-============-=================================
rc  zsh            5.9-6ubuntu2 amd64        shell with lots of features
```

Obtenemos el estado **`rc`**. El primer carácter (**`r`**) indica que el paquete debería haberse eliminado sin eliminar sus archivos de configuración. El segundo carácter (**`c`**) indica que los archivos de configuración del paquete existen, pero el paquete en sí no está instalado. En el caso de `zsh`, los ficheros de configuración de `/etc/zsh` pertenecen al paquete `zsh-common`, que se instaló como dependencia y sigue instalado.

Para saber dónde quedan ficheros relacionados con `zsh` (whereis no consulta el sistema de paquetes, sino los directorios habituales)

```
alumno@so-lab01:~$ whereis zsh
zsh: /etc/zsh /usr/share/zsh /usr/share/man/man1/zsh.1.gz
```

Si quieres borrarlo todo, incluidas configuraciones, tendrías que usar: (y, para retirar también `zsh-common` y `/etc/zsh`, `sudo apt autoremove --purge`)

```
alumno@so-lab01:~$ sudo apt purge zsh
alumno@so-lab01:~$ sudo apt --purge remove zsh  # equivalente a la anterior
alumno@so-lab01:~$ sudo apt autoremove --purge  # retira zsh-common y su
configuración
```

Ejercicios

1.- Instala los siguientes paquetes: `neofetch`, `tree`, `bat`, `curl`. Recuerda que antes de instalar nada siempre se debe actualizar la lista de paquetes y es recomendable actualizar el sistema. En Ubuntu, la orden del paquete bat se llama batcat, porque el nombre bat lo usa otro paquete (lo explica la descripción de apt show bat). En Ubuntu 26.04, `neofetch` no existe: usa `fastfetch`.

2.- Para cada paquete consulta la siguiente información usando los comandos `apt` y `dpkg`

- Comprueba si se ha instalado correctamente
- Qué ficheros ha instalado cada uno
- Para qué sirve
- Qué tamaño ocupa una vez instalado
- ¿Está mantenido directamente por la empresa Canonical? (ver la categoría a la que pertenece en el apartado Section de `apt show: universe/utils` indica la categoría `universe`; si no lleva prefijo, como «web», el paquete es de `main`)
- En qué sitio web puedes consultar más información sobre el proyecto (apartado Homepage)
- ¿De qué repositorio de paquetes se ha descargado? (consulta apt policy nombrePaquete o el apartado APT-Sources de apt show)
- Cómo se utiliza (utiliza *man*). Realiza pruebas simples de ejecución con cada uno para comprobar su funcionamiento

3.- Elimina alguno de ellos y verifica que ya no está instalado o si ha dejado archivos de configuración. En ese caso, indica al gestor de paquetes que los elimine también.